

Interaktivna računalna grafika

Dokumentacija rješenja laboratorijskih vježbi u modernom OpenGL-u

Ivan Dujmić

Vježba 1 – Crtanje linija na rasterskim prikaznim jedinicama.....	1
Vježba 2 – Crtanje i popunjavanje poligona	2
Vježba 3 – Prvi program u OpenGL-u	3
Vježba 4 – 3D tijela	4
Vježba 5 – Matrice modela, pogleda i projekcije	5
Vježba 6 – Uklanjanje skrivenih poligona	6
Vježba 7 – Bezierova krivulja	7
Vježba 8 - Sjenčanje	8
Vježba 9 – Teksture.....	9
Vježba 10 – Mape sjene.....	10

Vježba 1 – Crtanje linija na rasterskim prikaznim jedinicama

Zadatak prve vježbe bio je omogućiti crtanje linija na platnu klikovima miša koristeći Bresenhamov algoritam. Zatim je trebalo implementirati odsijecanje linija pravokutnikom.

Za početak su mi bile dostupne klase Graphics i Shader. Graphics sadrži funkcije za stvaranje i čišćenje prozora, obradu klikova miša te bojanje piksela 2D platna. Shader sadrži funkcije za kompajliranje, učitavanje i slanje uniformnih varijabli sjenčarima.

Iako sam najprije implementirao Bresenhamov algoritam s brojevima s pomičnim zarezmom i sam za kuteve od 0 do 45 stupnjeva, to sam zamijenio s verzijom koja koristi samo cjelobrojne tipove podataka te sam ga proširio da radi za sve kuteve. Navedeno sam implementirao u funkciji drawLine koja pojedine piksele boja koristeći Graphics::lightFragment. Funkcija mouseClicked stavlja točke u listu points.

Odsijecanje linija tako da prisutan bude samo dio unutar pravokutnika ostvareno je unutar drawLine funkcije. Inicijalni naivni pristup bio je provjeravanje odsijecanja za svaki piksel linije, a to je potom zamijenjeno Cohen Sutherlandovim algoritmom za odsijecanje linija. Za kodove Cohen Sutherland algoritma korištene su konstantne bitovne maske čija imena počinju s CC_ (Cohen code).

U funkciji main se nakon inicijalizacije programa u petlji izvodi sljedeće: čišćenje prozora, iscertavanje pozadine, iscertavanje pravokutnika za odsijecanje, iscertavanje svih linija, iscertavanje jednog dodatnog piksela po potrebi koji označava početak sljedeće linije.

Također je ostvareno bojanje linija ovisno o njihovoj duljini, a boju vraća funkcija calcLineColor čiju definiciju možemo zamijeniti i drugom formulom za odabir boje.

Dodatno unaprjeđenje bila bi implementacija anti-aliasinga koristeći Xiaolin Wu algoritam.

Vježba 2 – Crtanje i popunjavanje poligona

Cilj ove vježbe bio je omogućiti crtanje popunjenih poligona i nakon toga provjeravanje jesu li odabrane točke unutar ili izvan nacrtanog poligona.

Za crtanje linija (uključujući i onih s kojima se popunjava poligon) korištena je funkcija `drawLine` ostvarena u prethodnoj vježbi.

Funkcija `mouseClick` registrira lijevi i desni klik miša. U prvoj fazi izvođenja programa, dok poligon još nije zatvoren, lijevi klik miša dodaje novi vrh poligona te ga povezuje s prethodnim, a desni klik miša služi za zatvaranje poligona povezivanjem prvog i posljednjeg vrha te time program prelazi u drugu fazu. U drugoj fazi klikovi miša postavljaju točke koje su različito obojene ovisno o tome nalaze li se unutar ili izvan poligona.

Struktura `PolyElem` predstavlja jedan vrh poligona, a u sebi sadrži koordinate, koeficijente brida kojeg tvori sa sljedećim vrhom, zastavicu koja označava je li taj brid lijevi i pomoćnu varijablu koja pamti inverz duljine brida za potrebe računanja udaljenosti točke unutar poligona od brida poligona.

Struktura `Polygon` sadrži listu vrhova `PolyElem` te zastavice koje služe za pamćenje konveksnosti i "smjera" poligona.

Funkcija `calcAngle` računa posljednji stvoreni kut. Funkcija `calcPolyEdgeCoefs` računa koeficijente svih bridova, inverze njihovih duljina i označava ih kao lijeve ili desne. Funkcija `calcPolyConvex` provjerava je li poligon konveksan tako da za svaki brid provjerava je li sljedeće brid ispod ili iznad njega te ako za svaki vrijedi isto onda je konveksan, a inače je konkavan. Tako se može provjeriti i je li "smjer" poligona u smjeru kazaljke na satu ili suprotan.

Funkcija `fillPoly` ispunjava poligon tako da prvo odredi granični pravokutnik. Zatim za svaku horizontalnu liniju rastera unutar tog raspona računa lijevu i desnu granicu presjeka s rubovima poligona te prema tome iscrtava horizontalnu liniju kojom se poligon popunjava. Funkcije `calcDistPointPolygonInside`, `calcDistPolygon`, `calcDistPointEdge`, `calcRasterColor`, `calcFillColor` služe za bojanje površine unutar i izvan poligona u ovisnosti o udaljenosti od najbliže točke ovojnice poligona.

Poboljšanje koje se može postići u ovoj vježbi je vezano za popunjavanje poligona, a uključuje pamćenje aktivnog lijevog i desnog brida, i izmjenjivanje tek kada prijeđemo njegovu krajnju y vrijednost.

Vježba 3 – Prvi program u OpenGL-u

U ovoj vježbi kreće upoznavanje s OpenGL-om tako da se ostvari iscrtavanje susjednih ispunjenih trokuta različitih boja.

Crtanje trokuta ostvarena je na dva načina - koristeći "triangle strip" način crtanja te crtanjem svakog trokuta koristeći indekse triju vrhova koji čine pojedini trokut.

Klasa ProgramState sadrži trenutno stanje programa i neke mogućnosti za izmjenu tih stanja. Stanja su dimenzije prozora, pozicija miša, trenutna boja, boja koja se trenutno izmjenjuje i metoda crtanja trokuta. Također sadrži sve vrhove i boje te dodatno indeksirane vrhove radi druge metode iscrtavanja trokuta. Funkcije unutar ProgramState su addVertex za dodavanje novog vrha te handleKeyPress kojim se mijenja komponenta boje (R/G/B) koja se želi mijenjati i njen intenzitet.

Klasa Renderer sadrži sve potrebno za iscrtavanje na platno. Posjeduje attribute za sve sjenčare i odgovarajuće međuspremnik. Funkcija loadShader učitava sjenčar. Funkcija bufferData, ako je potrebno, prebacuje podatke za iscrtavanje na u OpenGL međuspremnik. Funkcija draw briše ekran i iscrtava obojeni kvadratić za biranje boje u kutu ekrana, trokute u odabranom načinu crtanja te liniju kroz zadane vrhove.

Prisutne su i funkcije za baratanje s unosima periferije.

Funkcija main inicijalizira prozor i platno te ulazi u radnu petlju u kojoj svaku iteraciju poziva bufferData i draw.

Korištena su tri različita sjenčara vrhova i fragmenata. Jedan je za kvadratić palete boja, taj prima uniformnu varijablu boje kojom popunjava kvadratić. Drugi je za vrhove trokuta, a prima poziciju i boju vrha. Zadnji sjenčar je za liniju koja redom povezuje odabrane vrhove, a poseban je po tome što u sjenčaru fragmenata za izlaz boje koristi funkciju koja ovisi o poziciji pojedine točke i protečenom vremenu.

Dobar dodatak ovoj vježbi bilo bi pred-prikazivanje trokuta koji bi se iscrtao kada bi kliknuli mišem na trenutnoj poziciji.

Vježba 4 – 3D tijela

Ova vježba je prva u nizu vježbi koje se nastavljaju jedna na drugu kako bi se ostvario jednostavan renderer za prikaz 3D scene u stvarnom vremenu. Počinje se s iscertavanjem žičanog modela tijela iz .obj datoteke s pomoću assimp biblioteke.

Klasa ResourceManager je singleton koji služi za dobivanje scena (modela) i sjenčara. Ako sjenčar nije već bio učitana onda ga učitava i sprema u mapu koristeći klasu Shader koja je opisana u prethodnim vježbama. Isto tako ima ulogu učitavanja scene, odnosno žičanog okvira objekta koristeći assimp.

Transform je bazna klasa objekata koja za sada sadrži poziciju, orijentaciju i faktor skaliranja objekta. Nasljeđuje ju klasa Object koja poznaje sve vrhove i lica svih objekata scene te sjenčar koji scena koristi. Njena klasa normalize poziva normalizaciju nad svim objektima unutar scene, a klasa render postavlja sjenčar i uniformnu varijablu boje te nad svim objektima poziva draw za iscertavanje. Ova konvencija imenovanja klase nije najbolja i bilo bi dobro dodati još jednu razinu apstrakcije.

Renderable je virtualna klasa koja zapravo predstavlja pojedine objekte scene (one koje pohranjuje klasa Object), a funkcije koje traži da njene podklase nasljeđuju su navedene normalize i draw. Konkretna implementacija Renderable klase je TriangleMesh koja sadrži sve vrhove objekta i listu indices koja indeksirano označava te vrhove u svrhu stvaranja pojedinih trokuta koji čine oplošje objekta. Funkcija normalize izračunava najmanju kocku u koju objekt stane te prema tome proporcionalno skalira objekt tako da stane u kocku dimenzije 2 i postavlja ga u središte koordinatnog sustava. Funkcija draw crta poligone žičanog okvira objekta.

Renderer dopušta dodavanje i uklanjanje instanci klase Object koje želimo iscertavati te nad njima omogućava poziv iscertavanja.

Glavni program kao argument prima ime objekta kojega želimo učitati te ga učitava, normalizira i na svakom frameu iscertava.

Korišteni sjenčar prima poziciju vrha te uniformnu boju koja se koristi za sve učitane objekte.

Vježba 5 – Matrice modela, pogleda i projekcije

Cilj vježbe 5 je ostvarenje kretanja i rotiranja kamere u 3D prostoru među učitanim žičanim okvirima objekata. U tu svrhu je implementirano primjenjivanje transformacija nad objektima na bazi matrica modela, pogleda i projekcije. Dvije su inačice ove vježbe - inačica a) transformacije ostvaruje preko glm biblioteke dok sam u inačici b) morao implementirati i samo dobivanje navedenih matrica.

Kameru predstavlja klasa Camera koja nasljeđuje klasu Transform, a dodaje varijable potrebne za određivanje perspektivne matrice i funkciju koja vraća tu matricu.

U klasu Transform sam dodao funkcije za dobivanje matrica modela i pogleda te funkcije za rotiranje objekta oko vlastitih osi, postavljanje orijentacije objekta te pomicanje s obzirom na vlastite osi i globalne osi. Ove funkcionalnosti ostvaruje pomoću klase MyGLM koja izvodi translaticije, skaliranje i rotiranje vektora te dobivanje matrica pogleda, perspektivne i ortogonalne projekcije. MyGLM je u inačici a) ostvarena koristeći glm, a u inačici b) bez glm biblioteke.

Sjenčar sada pri izračunu pozicije vrha redom množi matricu projekcije, matricu pogleda, model matricu i vrh proširen s homogenom koordinatom.

U glavnom programu su izvedene kontrole za pomicanje kamere, a bitno je korištenje vremenskih razmaka koje vraća FPSManager kako bi koraci pomicanja bili proporcionalni vremenskim razmacima između dvaju vremenskih točaka. Korištenjem tipki 0, 1, 2, 3 moguće je birati objekt nad kojim se želi kontrola pomicanja. Kontrola rotacije objekata je ostvarena tek u vježbi 10.

Zgodni dodatci ovoj implementaciji bili bi iscertavanje lokalnih i globalnih osi i mogućnost izmjenjivanja između ortogonalne i perspektivne projekcije. Kod bi mogao biti bolje organiziran i vjerojatno optimiziran (spajanjem objekata, praćenjem je li se dogodila ikakva promjena...).

Vježba 6 – Uklanjanje skrivenih poligona

Ova vježba fokusira se na uklanjanje skrivenih poligona koristeći više različitih algoritama. Svrha uklanjanja skrivenih poligona je poboljšanje performansi i vizualna točnost jer ne želimo vidjeti poligone koji bi trebali biti iza drugih poligona. Ovdje nije ostvareno uklanjanje skrivenih poligona jednog objekta relativno na druge objekte, već je to ostvareno u sljedećim vježbama koristeći Z-buffer.

U prvom od dva algoritma koja sam implementirao za uklanjanje skrivenih poligona radim provjeru nalazi li očište poslano preko uniformne varijable iznad ili ispod poligona. S obzirom na to da je redoslijed navođenja vrhova u datoteci objekta takav da možemo jednoznačno odrediti koja strana poligona je vanjska, a koja unutrašnja, množenjem jednadžbe ravnine poligona s točkom možemo odrediti njihov međusobni odnos. Između sjenčara vrhova i sjenčara fragmenata sada imamo i sjenčar geometrije koji prima sve vrhove jednog poligona tako da ga možemo čitavog ukloniti ako ne bi trebao biti vidljiv. Prije provjere relativne pozicije očišta nasprema licu treba primijeniti model matricu kako bi objekt stavili u globalni prostor.

Drugi algoritam se provodi u sustavu projekcije pa počinje tako da se najprije primjene sve tri matrice. Znamo je li lice stražnje ili prednje po "smjeru rotacije" točaka poligona u sustavu projekcije.

Još jedan način identifikacije stražnjih poligona je promatranjem kuta između vektora normale i vektora koji spaja očište sa središtem poligona. Ako je taj kut veći od 90 stupnjeva onda je poligon stražnji.

Vježba 7 – Bezierova krivulja

U vježbi 7 bilo je potrebno ostvariti crtanje aproksimacijske i interpolacijske Bezierove krivulje na bazi kontrolnog poligona određenog pritiscima određene tipke na različitim pozicijama kamere u prostoru. Također je bilo potrebno omogućiti kretanje kamere po interpolacijskoj krivulji. Dovoljno je ostvariti crtanje interpolacijske krivulje kroz posljednje četiri točke zbog toga što je potreban inverz matrice, a biblioteka glm podržava matrice do dimenzije 4×4 .

Za ostvarenje navedenog je dodana klasa `Curve` koja nasljeđuje `Renderable`, a sadrži točke iscrtkane linije i metodu `draw` za crtanje iste. Također je dodana klasa `BezierBuilder` s nekoliko korisnih funkcija. `makeApproximationCurve` prima kontrolne točke i, koristeći polinome Bernsteina te zadani broj segmenata koji određuje rezoluciju, određuje točke aproksimacijske krivulje zadane rezolucije. Funkcija `makeInterpolationCurveFromLast4` s pomoću `interpolationControlPointsFromLast4` i `makeApproximationCurve` vraća točke interpolacijske krivulje zadane rezolucije. To izvodi tako da iz zadnje četiri zadane interpolacijske točke izračuna kontrolne točke kubične Bezierova krivulje. Gradi Bernsteinovu matricu, invertira je i s pomoću tog inverza određuje takve Bezierove kontrolne točke da rezultirajuća aproksimacijska krivulja prolazi upravo kroz četiri zadane interpolacijske točke.

U mainu su funkcije za stvaranje i osvježavanje krivulja te su stvorene i "ravne krivulje" za prikaz globalnih osi.

Moguća poboljšanja ove vježbe su delegiranje izrade Bezierove krivulje sjenčaru geometrije i izrada interpolacijske krivulje za više od 4 točke.

Vježba 8 - Sjenčanje

U vježbi 8 se implementira sjenčanje objekta prema relativnom položaju izvora svjetlosti. Implementiran je Phongov model osvjetljenja uz tri različita načina izračuna intenziteta za svaki slikovni element: konstantno, Gouraudovo, Phongovo sjenčanje.

Dodane su dvije nove klase. Podatkovna klasa Material sadrži podatke o materijalu objekta, a za potrebe ove vježbe to su ambientColor, diffuseColor, specularColor i faktor shininess. Klasa Light nasljeđuje klasu Transform te također sadrži ambijentalnu, difuznu i spekularnu komponentu, ali za intenzitet osvjetljenja. Klasa Light predstavlja izvor svjetlosti kojeg možemo imati samo jednog zbog ograničenja moje implementacije klase Renderer.

Implementirao sam tri različita skupa sjenčara (vrhova, geometrijski, fragmenata), za navedena tri načina izračuna intenziteta. Prvo je konstantno sjenčanje, a ideja je da je čitav poligon jedne boje. Izračun intenziteta osvjetljenja za poligon odvija se u sjenčaru geometrije koji prima trokut po trokut te kroz uniformne varijable položaj očišta i informacije o izvoru svjetlosti i materijalu objekta. Poligon najprije translatiramo u prostor svijeta. Zatim računamo njegov centar kao aritmetičku sredinu vrhova, a na bazi njega ćemo odrediti osvjetljenje čitavog poligona. Potrebni su nam normala poligona, vektor koji povezuje poziciju izvora svjetlosti s centrom poligona, vektor koji povezuje poziciju očišta s centrom poligona i refleksija svjetlosti od poligon naspram njegove normale. Ambijentna komponenta je jednostavan umnožak dvaju ambijentnih komponenti. Difuzna komponenta je to veća što je kut upada svjetla više okomit na poligon. Spekularna komponenta također ovisi o položaju očišta i veća je što je manji kut između reflektirane zrake i vektora koji povezuje očište s centrom poligona.

Gouraudovo sjenčanje računa intenzitet u sva tri vrha te provodi interpolaciju za ostatak poligona. Za ovo je potrebno proslijediti i normale vrhova iščitane iz datoteke objekta. S obzirom na to da je jedina razina granulacije sada po vrhu, a ne po poligonu, nije nam potreban sjenčar geometrije, već se izračun odvija u sjenčaru vrhova.

Phongovo sjenčanje se odvija u sjenčaru fragmenata jer je ideja da interpoliramo između normala vrhova te da osvjetljenje računamo na bazi normala pojedinih interpoliranih točaka.

Vježba 9 – Teksture

Cilj ove vježbe bio je učitati i primijeniti teksturu objekta umjesto fiksne difuzne komponente materijala objekta.

Najprije je bilo potrebno u ResourceManageru iz datoteke objekta također učitati put do njegove difuzne teksture. Zatim sam dodao klasu Texture čiji konstruktor prima taj put i učitava datoteku teksture koristeći stbi_load. Format zapisa se postavlja ovisno o broju pročitanih kanala iz datoteke, a u testnom slučaju to je tri. Generira se tekstura te se postavlja omatanje s ponavljanjem tako da ako se dogodi da neka UV koordinata izađe izvan opsega $[0, 1]$, krene ponavljanje teksture. Za minimizirani prikaz teksturiranog objekta koristi se linearna interpolacija između dvaju susjednih mipmapa, a za magnificirani prikaz se koristi linearna interpolacija između susjednih piksela. Zatim se slika zapravo prebaci na GPU te se generiraju mipmape.

U klasi TriangleMesh je sada potrebno poslati i uniformnu varijablu difuzne teksture. Izmjena Phongovog sjenčara je jednostavna - sjenčar vrhova prima i proslijeđuje koordinate teksture vrha, a sjenčar fragmenata koristi boju s te koordinate u teksturi ako je dostupna umjesto difuzne boje materijala.

Bilo bi dobro još primijeniti mape ambijentalnog zasjenjivanja, sjajnosti, normala i visine.

Vježba 10 – Mape sjene

Cilj zadnje vježbe bio je ostvariti prikaz sjena, pri čemu se scena iscrtava dva puta: jednom iz pogleda izvora svjetlosti, a zatim iz očista promatrača.

Najprije je bilo potrebno napraviti dubinsku teksturu u koju se ne sprema boja fragmenta, nego samo njegova udaljenost od izvora svjetlosti. Zbog toga se dodaje framebuffer object (FBO) kako bi se rezultat prvog iscrtavanja spremio u teksturu umjesto da se prikaže na ekranu. Ta tekstura predstavlja dubinsku mapu scene gledano iz pozicije reflektora.

Za prvo iscrtavanje dodan je jednostavni depth sjenčar koji koristi projekcijsku matricu i matricu pogleda izvora svjetlosti. Rezultat tog iscrtavanja zapisuje se samo u teksturu dubine. Tako se dobiva informacija koji su fragmenti najbliži svjetlu i time osvijetljeni.

Drugo iscrtavanje je iz pogleda kamere, a sjenčaru se dodatno šalju i matrice izvora svjetlosti te prethodno izrađena mapa dubina. Sjenčar vrhova računa pozicije u koordinatnom sustavu svjetla, kako bi sjenčar fragmenata mogao provjeriti gdje se fragmenti nalaze u dubinskoj mapi.

U sjenčaru fragmenata se trenutna dubina fragmenta iz pogleda svjetla uspoređuje s dubinom spremljenom u depth mapi pa ako je fragment u sjeni se koristi samo ambijentalna komponenta osvijetljenja.

Koriste se i parametri svjetla `cutOffDeg` i `outerCutOffDeg` koji redom određuju širinu snopa svjetla maksimalnog intenziteta i širinu do koje onda snop postepeno slabi.

Ovakvo ostvarenje sjena rezultira s različitim artefaktima koje bi trebalo srediti. Bilo bi dobro proširiti program tako da podržava više izvora svjetlosti.